voice_live.py

For an AI-901 lab, I'd keep it **much simpler** than the full Learn sample. The purpose is:

Microphone

↓

Speech-to-Text

↓

Foundry Model

↓

Text Response

↓

Text-to-Speech

↓

Speaker

Save as:

voice_live.py

```
import azure.cognitiveservices.speech as speechsdk
from openai import OpenAI
# ----- User Input -----
speech_key = input("Speech Key: ").strip()
speech_region = input("Speech Region: ").strip()
project_endpoint = input("Project Endpoint: ").strip().rstrip("/")
api_key = input("API Key: ").strip()
deployment = input("Model Deployment Name: ").strip()
# ----- Speech Config -----
speech_config = speechsdk.SpeechConfig(
    subscription=speech_key,
    region=speech_region
```

```
)
speech_config.speech_recognition_language = "en-US"
recognizer = speechsdk.SpeechRecognizer(
    speech_config=speech_config
)
synthesizer = speechsdk.SpeechSynthesizer(
    speech_config=speech_config
)
# ----- Foundry Client -----
client = OpenAI(
    api_key=api_key,
    base_url=f"{project_endpoint}/openai/v1/"
)
print("\nVoice Chat Ready")
print("Say 'goodbye' to quit.\n")
# ----- Conversation Loop -----
while True:
    print("Listening...")
    result = recognizer.recognize_once()
    if result.reason != speechsdk.ResultReason.RecognizedSpeech:
        print("Could not understand audio.\n")
        continue
    user_text = result.text.strip()
    print(f"\nYou: {user_text}")
    if "goodbye" in user_text.lower():
        print("Ending session.")
        break
    try:
        response = client.responses.create(
```

```
        model=deployment,
        input=user_text
    )
    ai_text = response.output_text
    print(f"\nAI: {ai_text}\n")
    synthesizer.speak_text_async(ai_text).get()
  except Exception as ex:
    print(f"\nError: {ex}\n")
```

Install Packages

pip install openai azure-cognitiveservices-speech

Run

python voice_live.py

Student Inputs

Speech Key

Speech Region

Project Endpoint

API Key

Model Deployment Name

Example:

Speech Region:

eastus

Deployment:

gpt-5-mini

What This Demonstrates

✅ Speech Recognition

Speech

↓

Text

✅ Foundry Model Call

Text

↓

GPT-5-mini

↓

Response

✅ Speech Synthesis

Response

↓

Audio

✅ Basic Voice Agent Architecture

User

↓

Microphone

↓

Azure Speech

↓

Foundry Model

↓

Azure Speech

↓

Speaker

Future Enhancements

After students get this working, add one feature at a time:

voice_live.py

↓

Conversation History

↓

RAG Search

↓

Foundry Agent

↓

MCP Tools

↓

Realtime Voice Live

This keeps the first lab focused on the core idea:

"I can speak to Azure Speech, send the recognized text to a Foundry model deployment, and hear the response spoken back."